

ClaudeSkills / Agent Workflow Kit 設計書 v0.1

1. 背景

LLMエージェントは、アイデア出し、設計、仕様整理、ドキュメント作成などの「拡張フェーズ」では非常に強い。

一方で、実装後のバグ修正、テスト修正、既存コードへの小改修、差分レビューなどの「収束フェーズ」では、以下の問題が起きやすい。

- 既存仕様を壊す
- 目的外のリファクタリングを混ぜる
- テスト期待値を実装都合で変更する
- 過去の会話文脈を誤って参照する
- 途中で変更された仕様を混同する
- 変更範囲が広がる
- 失敗時に原因分析なしで連続修正する
- スキルやサブエージェントが期待通りに自動発動しない

特に、LLMはOR的な発想、つまり候補を広げる処理には強いが、AND的な処理、つまり複数の制約を同時に満たす処理には弱い。

そのため、収束フェーズでは、LLMの能力だけに頼るのではなく、以下のような制御を組み合わせる必要がある。

- AGENTS.md / CLAUDE.md による行動ルール
- Session Brief による短期状態固定
- プロンプトテンプレートによる手順固定
- git diff による実変更レビュー
- サブエージェントまたは別セッションによる独立レビュー
- shell script によるローカルゲート
- Git Hook による pre-commit / pre-push 制御
- 小さいコミットによる変更範囲の抑制

2. 目的

この設計では、既存の ClaudeSkills リポジトリを拡張し、Claude Code / Codex 共通で利用できる「収束フェーズ制御キット」を追加する。

目的は以下である。

1. Claude Code / Codex の両方で使える共通のエージェント運用ルールを整備する
2. スキルの自動発動に依存せず、明示的に呼び出せるプロンプト・擬似コマンドを整備する
3. 収束フェーズを SDD + TDD + diff review + gate の流れで安定化する
4. サブエージェントレビューでは親コンテキストを前提にせず、SESSION_BRIEF と git diff を根拠にする
5. Git Hook と shell script により、LLMの外側で最低限のローカルゲートを実現する
6. 将来的なりポジトリ名変更にも耐えられる汎用的な構成にする

3. 基本方針

3.1 Claude 専用にならない

既存リポジトリ名は ClaudeSkills だが、今後は Claude Code だけでなく Codex でも利用する。

そのため、Claude固有機能だけに依存しない。

中心に置くものは以下とする。

- Markdown
- shell script
- Git Hook
- git diff
- AGENTS.md
- SESSION_BRIEF.md
- prompts/*.md

Claude Code の Skill、Subagent、Slash Command は補助として扱う。

Codex の AGENTS.md、Subagent、Slash Command、approval / sandbox も補助として扱う。

3.2 スキルの自動発動に依存しない

スキルは有用だが、期待した場面で自動発動しない場合がある。

そのため、以下の方式を採用する。

- 常時ルールは AGENTS.md / CLAUDE.md に書く
- 詳細手順は prompts/*.md に書く
- 今回の作業状態は SESSION_BRIEF.md に書く
- 実行時は擬似コマンドまたは明示指示で prompts/*.md を呼ぶ
- 強制的な検査は shell script / Git Hook に置く

3.3 収束フェーズでは SDD + TDD を使う

簡単なバグ修正でも、以下の順番で進める。

1. Spec: 現在の挙動、期待仕様、不一致点を整理する
2. Test: 失敗テストを作る、または既存テストで再現する
3. Implement: テストを通すための最小差分で実装する
4. Review: git diff を確認する
5. Gate: pre-commit-gate.sh を実行する
6. Commit: 小さい単位でコミットする

3.4 レビュー担当は親コンテキストを引き継がない

レビュー担当サブエージェント、または別セッションのレビュー担当は、実装担当の会話履歴や推論を前提にしない。

レビューの根拠は以下に限定する。

- AGENTS.md
- agent/SESSION_BRIEF.md
- git status
- git diff
- 必要な場合のみ対象ファイルの現在内容

レビュー担当はコード変更しない。

4. 制御レイヤー

4.1 入力・文脈制御レイヤー

LLMの振る舞いを制御する層。

対象:

- AGENTS.md
- CLAUDE.md
- prompts/*.md
- 擬似コマンド
- 通常プロンプト

役割:

- どう振る舞うかを定義する
- 収束フェーズの作業手順を固定する
- レビュー担当の禁止事項を定義する
- SDD + TDD の順序を定義する

4.2 状態固定レイヤー

今回の作業状態を固定する層。

対象:

- agent/SESSION_BRIEF.md
- briefs/*.template.md

役割:

- 今回の目的を固定する
- 現在の確定仕様を固定する
- 対象・非対象を明示する
- 禁止事項を明示する
- 過去の会話よりも SESSION_BRIEF を優先させる

4.3 LLM拡張機能レイヤー

Claude Code / Codex の機能を補助的に使う層。

対象:

- Claude Skill
- Claude Code Subagent
- Claude Code Slash Command
- Codex Subagent
- Codex Slash Command
- Model Switching

役割:

- 必要に応じて専門化された作業を呼び出す
- レビュー担当を分離する
- 拡張フェーズと収束フェーズでモデルを切り替える

注意:

- 製品固有機能には依存しすぎない
- 共通基盤は Markdown / sh / Git に置く

4.4 ローカル実行制御レイヤー

LLMの外側で機械的に確認する層。

対象:

- agent/gates/pre-commit-gate.sh
- agent/gates/check-sensitive-files.sh
- agent/gates/check-large-files.sh
- agent/gates/check-diff-basic.sh

役割:

- 秘密ファイルの混入を防ぐ
- 巨大ファイルの混入を防ぐ
- staged diff の基本確認を行う
- 空白エラーを検出する
- Git Hook から呼び出せる共通ゲートにする

4.5 Git制御レイヤー

Gitの仕組みで制御する層。

対象:

- agent/hooks/pre-commit
- agent/hooks/pre-push
- git diff
- git diff --cached
- git status
- 小さいcommit

役割:

- commit 前に gate を自動実行する
- main / master への直接 push を防ぐ
- 実際の差分を真実としてレビューする
- 1目的1コミットで差分を小さくする

4.6 レビュー制御レイヤー

実装担当とは別視点で確認する層。

対象:

- prompts/diff-review.md
- prompts/subagent-review.md
- prompts/pr-review.md
- Claude Code Subagent
- Codex Subagent
- 別セッションレビュー

役割:

- 目的外変更を検出する
- テスト期待値の都合のよい変更を検出する
- 対象外ファイル変更を検出する
- 実装担当の文脈に引きずられないレビューを行う

4.7 GitHub制御レイヤー

リモート側で最終的に守る層。

初期段階では必須ではない。

将来的な対象:

- Pull Request
- Branch protection
- Required review
- GitHub Actions
- Required checks

役割:

- local gate をすり抜けた変更をリモートで止める
- main を Prod として保護する
- PRレビューを必須化する

5. 推奨ディレクトリ構成

既存リポジトリ内に、共通エージェント運用部分を追加する。

初期案:

```
ClaudeSkills/  
  README.md  
  
  test-orchestrator/  
  code-review/  
  cowork-chrome-launcher/  
  agents/  
  
  common/  
    README.md  
  
    rules/  
      AGENTS.base.md  
      CLAUDE.base.md  
  
    prompts/  
      converge-bugfix.md  
      diff-review.md  
      subagent-review.md  
      failure-analysis.md  
      pr-review.md  
  
    briefs/  
      SESSION_BRIEF.template.md  
      BUGFIX_BRIEF.template.md  
  
    gates/  
      pre-commit-gate.sh  
      check-sensitive-files.sh  
      check-large-files.sh  
      check-diff-basic.sh  
  
    hooks/  
      pre-commit  
      pre-push  
  
    setup/  
      setup-hooks.sh  
  
    project-templates/  
      AGENTS.md  
      CLAUDE.md  
      Makefile
```

5.1 既存部分

既存の以下は維持する。

```
test-orchestrator/  
code-review/  
cowork-chrome-launcher/  
agents/
```

5.2 新規追加部分

新規に `common/` を追加する。

`common/` は Claude / Codex 共通で使えるエージェント運用キットとする。

6. 擬似コマンド

Claude / Codex 共通で使うため、正式な slash command ではなく、擬似コマンドを定義する。

初期コマンド:

```
@converge-bugfix  
@diff-review  
@subagent-review  
@gate  
@failure-analysis
```

6.1 @converge-bugfix

用途:

- 収束フェーズのバグ修正を開始する

動作:

- AGENTS.md を読む
- agent/SESSION_BRIEF.md を読む
- prompts/converge-bugfix.md の手順に従う
- 最初は Phase 1: Spec のみ行う
- コード変更は禁止

例:

```
@converge-bugfix ログイン後に /dashboard に遷移しない問題。まず Phase 1: Spec のみ。コード変更は禁止。
```

6.2 @diff-review

用途:

- 実装後の git diff をレビューする

動作:

- git status を確認する
- git diff を確認する
- SESSION_BRIEF と差分を照合する
- コード変更は禁止

例:

@diff-review 今回の目的と無関係な変更がないか確認。コード変更は禁止。

6.3 @subagent-review

用途:

- レビュー専用サブエージェント、または別セッションで独立レビューを行う

動作:

- 親会話の文脈を前提にしない
- SESSION_BRIEF と git diff を根拠にする
- OK / WARNING / BLOCKER で判定する
- コード変更は禁止

例:

@subagent-review SESSION_BRIEF.md と git diff だけを根拠にレビュー。親会話の文脈は前提にしない。コード変更は禁止。

6.4 @gate

用途:

- ローカル gate を実行する

動作:

- ./agent/gates/pre-commit-gate.sh を実行する
- 失敗した場合は、追加修正前に原因分析する

例:

@gate 失敗した場合は、追加修正の前に原因分析だけしてください。

6.5 @failure-analysis

用途:

- gate、テスト、レビュー失敗時に原因分析を行う

動作:

- すぐに追加修正しない
- 失敗原因を整理する
- 最小修正方針を出す
- コード変更は禁止

例:

@failure-analysis pre-commit-gate が失敗しました。追加修正の前に原因分析のみ行ってください。

7. 収束フェーズ標準フロー

Step 1: Session Brief 作成

作業前に `agent/SESSION_BRIEF.md` を作成または更新する。

含める内容:

- 作業モード
- 今回の目的
- 現在の確定仕様
- 現在の問題
- 対象
- 非対象
- 禁止事項
- 検証方法

Step 2: Spec

LLMに `@converge-bugfix` を指示する。

この段階ではコード変更禁止。

出力させるもの:

- 現在の挙動
- 期待仕様
- 不一致点
- 影響範囲
- テストで固定すべき条件
- 修正対象候補
- 変更予定ファイル

Step 3: Test

ユーザーが許可したらテスト作成に進む。

ルール:

- 仕様に基づいて失敗テストを作る
- 既存テストで再現できる場合は新規追加しない
- 実装コードは変更しない
- テスト期待値を実装都合で変更しない

Step 4: Implement

ユーザーが許可したら実装に進む。

ルール:

- 最小差分で実装する
- テスト期待値を勝手に変更しない
- 対象外ファイルを変更しない
- 関係ないリファクタリングをしない

Step 5: Diff Review

実装後、`@diff-review` を実行する。

確認観点:

- 今回の目的に必要な変更か
- 仕様と関係する変更か
- 目的外変更がないか
- 最小差分か
- テスト期待値を都合よく変更していないか

Step 6: Subagent Review

必要に応じて `@subagent-review` を実行する。

レビュー担当は、親会話の文脈を前提にせず、SESSION_BRIEF と git diff だけを根拠にする。

Step 7: Gate

`@gate` または直接 `./agent/gates/pre-commit-gate.sh` を実行する。

失敗時は `@failure-analysis` に進む。

Step 8: Commit

1目的1コミットで commit する。

`pre-commit` hook により gate が自動実行される。

8. Git Hook 方針

8.1 pre-commit

中心的に使う。

実行内容:

- git diff --cached --check
- check-sensitive-files.sh
- check-large-files.sh
- check-diff-basic.sh

8.2 pre-push

補助的に使う。

初期用途:

- main / master への直接 push 禁止

9. モデル切り替え方針

拡張フェーズ

最新・高性能モデルを使う。

用途:

- アイディア出し
- 設計比較
- 仕様整理
- 大きな方針検討

収束フェーズ

安定モデル、中位モデル、または1世代前モデルを使う。

用途:

- 最小差分修正
- diff確認
- 軽いバグ修正
- gate前確認

難所

収束フェーズ中でも、原因不明の失敗や複雑な依存関係がある場合は、最新・高性能モデルに一時エスカレーションする。

ただし、その場合もコード変更は禁止し、原因分析だけを行う。

10. 初期実装スコープ

v0.1 では以下を実装する。

```
common/  
  README.md  
  
rules/  
  AGENTS.base.md  
  CLAUDE.base.md  
  
prompts/  
  converge-bugfix.md  
  diff-review.md  
  subagent-review.md  
  failure-analysis.md  
  
briefs/  
  SESSION_BRIEF.template.md  
  
gates/  
  pre-commit-gate.sh  
  check-sensitive-files.sh  
  check-large-files.sh  
  check-diff-basic.sh  
  
hooks/  
  pre-commit  
  pre-push  
  
setup/  
  setup-hooks.sh
```

以下は v0.1 では任意または将来対応とする。

- GitHub Actions
- Branch protection
- allowed-files.txt による厳格な対象ファイル制御
- Claude Code の正式 slash command 化
- Codex の正式 slash command 対応
- Cowork 用 .skill 化
- npm package / installer 化

11. 未決事項

11.1 リポジトリ名

現状は `ClaudeSkills`。

将来的には以下のような名前に変更候補がある。

- AgentOpsKit
- AgentWorkflowKit
- AgentControlKit
- LLMAgentOps
- AIAgentWorkflow

11.2 配布方式

未決。

候補:

- 各プロジェクトへコピー
- symlink
- Git submodule
- setup script による展開
- テンプレート生成

初期は、既存運用に合わせて symlink またはコピーでよい。

11.3 擬似コマンド記法

初期は `@xxx` とする。

将来的に Claude Code 用には slash command 化を検討する。

11.4 Git Hook の厳しさ

初期は以下とする。

- 秘密ファイル: block
- 巨大ファイル: block
- 空白エラー: block
- テストファイル変更: warning
- 目的外変更: warning

将来的に必要なであれば厳格化する。

12. 結論

既存の ClaudeSkills リポジトリを拡張し、`common/` 配下に Claude / Codex 共通のエージェント運用キットを追加する。

この設計では、スキルの自動発動に依存せず、以下の組み合わせで収束フェーズを安定化する。

- AGENTS.md / CLAUDE.md
- SESSION_BRIEF.md
- prompts/*.md
- 擬似コマンド

- diff review
- subagent review
- shell gate
- Git Hook
- 小さい commit

v0.1 の目標は、まず Markdown と shell script を整備し、ローカルリポジトリで Codex に実装させられる状態にすることである。